In addition to running on each platform, one of the selling points of .NET is your ability to write and compile only once. Your application is compiled to Intermediate Language (IL) code, which is a platform-independent format. If a target system has the .NET 7 runtime installed, you can run compiled IL from any platform. You can develop on a Mac or a Windows machine, for example, and deploy *exactly the same files* to your production Linux machines. This compile-once, run-anywhere promise has finally been realized with ASP.NET Core and .NET 7.

TIP You can go one step further and package the .NET runtime with your app in a so-called *self-contained deployment* (SCD). This way, you can deploy cross-platform, and the target machine doesn't even need .NET installed. With SCDs, the generated deployment files are customized for the target machine, so you're no longer deploying the same files everywhere in this case.

Many of the web frameworks available today use similar well-established *design patterns*, and ASP.NET Core is no different. Ruby on Rails, for example, is known for its use of the MVC pattern; Node.js is known for the way it processes requests using small discrete modules (called a *pipeline*); and dependency injection is available in a wide variety of frameworks. If these techniques are familiar to you, you should find it easy to transfer them to ASP.NET Core; if they're new to you, you can look forward to using industry best practices!

NOTE *Design patterns* are solutions to common software design problems. You'll encounter a pipeline in chapter 4, dependency injection in chapters 8 and 9, and MVC in chapter 19.

Whether you're new to web development generally or only with .NET, ASP.NET Core provides a rich set of features with which you can build